
Práctica 4. Planificación de caminos con mapa

Alejandro Otero Vázquez y Mario Bravo López

Fecha de realización: 13 de enero de 2025

El estado del robot en el momento en que se afrontó esta práctica era el de un móvil capaz de moverse en siguiendo una trayectoria punto a punto esquivando obstáculos imprevistos. Los principales problemas que presenta el robot están dados por la odometría que afecta a su trayectoria. Errores mínimos en la trazada de un giro, por ejemplo, se van acumulando a medida que el robot recorre más distancia dando lugar a desviaciones respecto a la trayectoria perceptibles. Aunque este no sea el reto de esta práctica, sí que se mantiene como un objetivo secundario a mejorar del robot. Será fundamental reducir los errores de odometría para que la correlación de la información del mapa que seguirá el robot y el camino recorrido se mantenga.

El objetivo de esta práctica es utilizar la plataforma del robot móvil construida en las sesiones anteriores para realizar tareas más complejas. En esta práctica se planificarán trayectorias para desplazarse a un punto determinado a partir de un mapa dado. Una vez planificada la ruta a seguir mediante uno de los métodos vistos en clase, se realizará un seguimiento de la trayectoria planificada mientras se verifica que no se encuentran obstáculos imprevistos

Experimento 1. Planificación de la trayectoria más adecuada a partir de un mapa dado.

Una vez realizada la introducción al problema que se quiere abordar, empezaremos a explicar de manera conceptual, y mas en adelante con código, la forma en la cual se ha implementado la esquivas de obstáculos dado un mapa con los mismos.

La idea de partida es bastante intuitiva, dado un punto de origen (x_0, y_0) y un punto al que se quiere llegar (x_f, y_f) se barre el mapa de tal manera que si en la trayectoria para alcanzar el punto final se detecta un vértice el robot tomara este punto como uno mas del camino necesario a seguir para alcanzar el punto final. Este proceso se realiza de manera iterativa hasta que el punto en el que se encuentra el robot corresponde al punto final. Aunque en primera instancia pueda parecer relativamente sencillo, implementar este algoritmo requirió bastantes horas de trabajo, pues aparecían singularidades que hasta entonces no se barajaban.

Funciones auxiliares.

La idea como ya se ha comentado es la de generar un camino que el robot sea capaz de seguir, como se hacía en la práctica 3 en el tercer apartado, en el cual se le suministraba un camino del tipo: $\text{camino} = \{x_0, y_0, x_1, y_1, \dots, x_f, y_f\}$ y el robot era capaz de seguirlo de manera precisa, reajustando la trayectoria en caso de que por fallos de odometría el robot se desviará mucho de la trayectoria esperada. Por lo tanto, usando el software de la anterior práctica, con toda la parte de actuadores y sensores ya implementada, necesitamos añadir una nueva funcionalidad con la cual dada un mapa sea capaz de generarnos el camino que hay que seguir para partiendo de un punto dado por consola (x_0, y_0) , alcanzar un punto final (x_f, y_f) , evitando los obstáculos del mapa proporcionado al robot. Para conseguir esto se hizo uso de la función `calculo_camino()`, cuyo funcionamiento podemos entender de la siguiente manera:

Algorithm 1: calculo_camino

Input: Punto de partida, Punto final, Mapa, Posición actual, tamaño del camino

Output: Array con las coordenadas del camino.

*Se inicializa el vector camino y se le asocia memoria dinámica, conocidos punto final e inicial y se evalúa el tipo de movimiento a realizar con la función: **funcion_aux_tipo_movimiento()** ;*

while *El último punto del camino no es el punto final* **do**

 Inicializamos a cero variable *contador_vertices*;

 Con ayuda de **funcion_aux_tipo_movimiento()**, se determina el tipo de movimiento que se ha de realizar para alcanzar el punto final;

 En la mayoría de casos se busca recursivamente un vértice disponible cercano al punto final, para ello se calculan los desplazamientos en línea recta en el eje x e y necesarios para alcanzar el punto final.;

 En función del signo que tengan Δx y Δy , se busca en el mapa barriendo las filas de arriba hacia abajo ($\Delta y < 0$) o de abajo a arriba ($\Delta y > 0$), de izquierda a derecha ($\Delta x > 0$) o a la inversa ($\Delta x < 0$).;

if al realizar el barrido *mapa==2* **then**

Se ha localizado un vértice en la dirección del punto final;

 Si para poder alcanzar este vértice el robot atraviesa tanto horizontal como verticalmente un objeto ese vértice se descarta. Uso de *break* para saltarse esa fila pues aunque se encuentre un vértice será inaccesible;

 Si no se encuentra objeto entre medias, el nuevo vértice se evalúa con **optimizacion_vertices()**, y si es óptimo para alcanzar el punto final se guarda en *camino* usando **save_point()** y *contador_vertices* pasa a incrementar, dando a entender que no se puede llegar de manera directa al punto final sin antes pasar por un vértice

end

if *contador_vertices==0* **then**

No se han detectado vértices entre punto final y punto en el que se encontraba el robot;

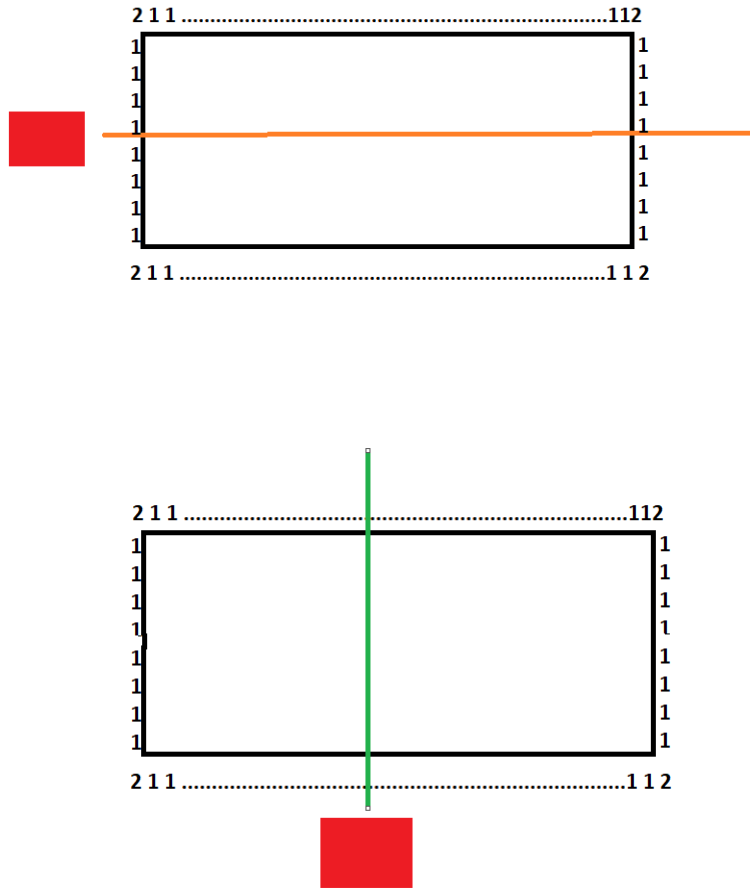
 Consecuentemente se puede ir directamente al punto final sin atravesar ningún obstáculo, se usa **save_point** para actualizar *camino* con el punto final

end

end

Este algoritmo se ha simplificado bastante, pues tiene muchas condiciones especiales que se intentaran aclarar ahora con las siguientes funciones auxiliares, por ende el lector debe de tener en cuenta que de manera recursiva se hará referencia a esta primera función.

Como ya hemos comentado *funcion_aux_tipo_movimiento* es una función clave para poder calcular este camino, y que se llama al principio de *calculo_camino*. El principal objetivo de esta función recae en diferenciar dos casos especiales de desplazamiento, pues tal y como estaba planteado *calculo_camino* nos daban problemas. Estos corresponden a cuando se intentaba realizar un desplazamiento completamente vertical u horizontal y en mitad se encontraba un objeto. Pues la forma que tiene de barrer el mapa *calculo_camino* es ir evaluando en cada fila del mapa, entre la columna en la que se encuentra el robot y la columna del punto final, para poder encontrar un vértice. El problema reside en que si se quiere realizar un desplazamiento completamente horizontal u vertical y por medio hay un objeto:



El robot entra en un bucle y no es capaz de superar el objeto, pues en el barrido en busca de vértices que realiza no es capaz de encontrar ninguno de apoyo para llegar a su destino final. Es en estos casos donde entra en juego:

Algorithm 2: funcion_aux_tipo_movimiento

Input: Punto de partida, Punto final, Mapa, Posición actual

Output: Un entero que activa rutinas de esquiva de obstaculos, deteccion_obstaculos.

Se evaluan cuatro posibles caminos conflictivos;

if $\Delta x == 0 \ \&\& \ \Delta y < 0$ **then**

Caso en el que se vaya a realizar un desplazamiento vertical descendente;

 Si al hacer el barrido de filas se encuentra un objeto, $\text{mapa}[i][x] == 0$ y $\text{mapa}[i+1][x] == 1$, entonces la variable *deteccion_obstaculos*=1;

end

else if $\Delta y == 0 \ \&\& \ \Delta x < 0$ **then**

Caso similar pero en distinta direccion;

 Si al hacer el barrido de columnas se encuentra un objeto, $\text{mapa}[y][i] == 0$ y $\text{mapa}[y][i+1] == 1$, entonces la variable *deteccion_obstaculos*=2;

end

else if $\Delta y == 0 \ \&\& \ \Delta x > 0$ **then**

Se actúa de manera igual, deteccion_obstaculos=2;

end

else if $\Delta x == 0 \ \&\& \ \Delta y > 0$ **then**

Se actúa como en el primer caso, si se detecta obstaculo, deteccion_obstaculos=1;

end

En caso de no detectarse ningún obstáculo de por medias la salida dará 0. Esta función da como respuesta un entero que será evaluado por `calculo_camino`, en función del entero que proporcione la función auxiliar, `calculo_camino` tendrá un comportamiento normal, caso en el que *deteccion_obstaculos*=0, u activará una de las rutinas de `esquiva_vertical()` en el caso que *deteccion_obstaculos*=1 o se activará la rutina `esquiva_horizontal()` en el caso que *deteccion_obstaculos*=2. Aunque en general la mayoría de veces estas rutinas no se utilizan, pues es solo en condiciones muy específicas donde es necesario rodear un obstáculo.

Volviendo a la función principal `calculo_camino` cuando se detecta un vértice al realizar el procesamiento iterativo celda por celda se usa la función `optimizacion_vertice()`, esta función es de vital importancia pues evalúa si el vértice encontrado nos interesa añadirlo al camino, como se verá a continuación. El principal parámetro que determinará si el nuevo vértice localizado se añade o no al camino es si la distancia de este al punto final es menor que la distancia a la que se encuentra el robot del punto final. La estructura que sigue es la siguiente:

Algorithm 3: `optimizacion_vertices`

Input: Coordenadas x e y del vértice detectado, Punto final, Posición actual, tamaño del camino, contador de vertices

Output: Un entero que refleja el nuevo tamaño del camino.

Se compara la distancia desde el nuevo vértice al punto final con, la distancia actual del robot al punto final;

if *distancia_vertice*<*distancia_actual* **then**

Se actualiza la posición actual del robot a la del vértice, usando punteros;

Además se incrementa el tamaño del camino en dos unidades, este nuevo valor sera el que retorne la función, y también aumenta la variable *contador_vertices*, al hacer esto evitamos que en `calculo_camino()` se entre en la condición de no haber detectado un vértice.

end

Evidentemente si el vértice detectado no cumple la condición de disminuir la distancia relativa al punto final ni se aumenta *contador_vertices*, ni el tamaño del camino. Una vez aclarado bajo que parámetros se incrementará el número de puntos que tiene el camino que debe seguir el robot, solo queda explicar como conseguimos añadir un nuevo punto al camino del robot. Este propósito lo desempeña la función `save_point`. La cual se encargará de almacenar el vértice detectado y previamente evaluado por `optimizacion_vertices`, su estructura es simple:

Algorithm 4: `save_point`

Input: camino, nuevo tamaño del camino, punto a introducir en el camino

Output: camino

Esta función se encarga principalmente de almacenar el nuevo vértice en el camino;

Primero se aumenta el tamaño del array usando `realloc()` y luego se almacena en las nuevas posiciones del array.

Esta función es muy importante que transforme las coordenadas de las celdas del mapa en distancias en mm, pues a fin de cuentas tal y como se tiene diseñado el sistema de seguimiento de un camino de la práctica anterior este interpreta los puntos en mm. Estas tres serían las funciones más importantes, siendo evidentemente `calculo_camino`, el eje del control del robot y las otras ramificaciones que optimizan su funcionamiento.

Una vez se tiene calculado el camino toca pasar este camino a instrucciones que el robot sea capaz de interpretar y ejecutar, esta segunda parte que ha de realizar la función `calculo_camino`, se estudiará en la siguiente parte del informe para no sobrecargar mucho al lector con las funciones que realiza `calculo_camino`.

Experimento 2. Seguimiento de trayectoria y esquivas de obstáculos.

Llegados a este punto se ha conseguido que el robot sea capaz de planificar y seguir una trayectoria a partir de un mapa sobre el que se han dibujado obstáculos. El objetivo ahora es que durante la marcha el mapa se modifique si los sensores de distancia detectan algún obstáculo. El algoritmo de esquivas es distinto del que se ejecutaba cuando la trayectoria del robot no sigue un mapa. Para ello se ha implementado una recalculación de la trayectoria in-situ en el momento en que se ha detectado un obstáculo a partir del mapa modificado.

Anteriormente la función `motorMoveRelative()` encargada de mover el dispositivo en línea recta, ejecutaba dentro de sí la función de esquivas de obstáculos `motorDodgeRoutine()` si se verificaba la condición de que los sensores medían una distancia menor a la umbral. El algoritmo de planificación de caminos y movimiento se recoge en su totalidad en la función `calculo_camino()`. Esta función tiene la siguiente estructura:

Algorithm 5: Calculo_camino

Input: Punto de partida, Punto final, Mapa, Posición actual

Output: Array con las coordenadas del camino.

Se eliminan los puntos que pudieran estar almacenados en camino.;

Reserva de memoria dinámica para almacenar el array de camino.;

while El último punto del camino no es el punto final **do**

 Cálculo del vértice disponible más cercano al punto final.;

 Reasignación de memoria dinámica al camino para almacenar el nuevo punto.;

end

i←0;

k almacena el tamaño del array camino, que almacena las coordenadas de los puntos calculados a partir del mapa. **while** (*i* < *k* || *k* != 0) **do**

Se modifican las posiciones del siguiente punto al que irá el robot siguiendo las posiciones almacenadas en el camino.

modifyCoordinates(camino[i],camino[i+1],nextPosition);

i←**i**+2;

Dadas dos posiciones, actual y siguiente, se calcula un ángulo de giro y una distancia a recorrer para que el robot se sitúe en la nueva posición.

generateInstructions(actualPosition, nextPosition, k);

end

Si se detecta un obstáculo k será igual a 0, ya no hay camino.

if Obstáculo detectado (*k*==0). **then**

Se modifica el mapa para dibujar el obstáculo detectado.

modifyMapper(mapa,punto detectado,dimensiones obstáculo);

Se vuelve a llamar a la función calculo_camino con el mapa modificado y la posición actual como entradas.

calculo_camino(origen, final, mapa, actualPosition);

end

else

 Se libera la memoria dinámica de que se ha asignado durante el algoritmo;

 return **camino**;

end

Fin del algoritmo;

Funciones auxiliares.

La función de `calculo_camino` acude durante su ejecución a un par de funciones auxiliares relativas a la detección y esquivas de obstáculos. Cuando un obstáculo es detectado, el programa accede a la dirección de memoria que almacena el mapa para añadir un obstáculo. El algoritmo de modificar mapa sigue la siguiente

estructura:

Algorithm 6: modifyMapper

Input: Mapa, Centro, Dimension

Output: Void.

Se recorren con un doble bucle las celdas ocupadas por el obstáculo para modificar sus valores.;

```
for i in x de las celdas ocupadas por el obstáculo do
    for j in y de las celdas ocupadas por el obstáculo do
        Mapa[i · Mapa->columns + j];
        if i == x de un vértice then
            Mapa[(Centro->y - Dimension->y)·Mapa->columns+i];
            Mapa[Centro->y·Mapa->columns+i];
        end
    end
end

for i in x de las celdas ocupadas por el obstáculo do
    for j in y de las celdas ocupadas por el obstáculo do
        printf(Mapa[i · Mapa->columns + j]);
    end
end
```

Fin del algoritmo;

Cuando el algoritmo ha calculado finalmente los puntos que definen la trayectoria del robot dado un mapa sobre el que se han dibujado los obstáculos (detectados y por defecto) se llama a `generateInstructions()`. Esta función calcula el ángulo de giro y la distancia que debe recorrer el robot para llegar a una posición cualquiera teniendo en cuenta las coordenadas en las que se encuentra el móvil en cada instante. Esta función tiene en cuenta también si durante el movimiento se ha detectado algún obstáculo. La estructura de la función se describe a continuación:

Algorithm 7: generateInstructions

Input: Posición actual, Posición objetivo, Distancia de seguridad, *Número de puntos del camino.

Output: Void.

*La posición actual se almacena en **ActualPosition**. Análogamente, la posición objetivo se almacena en **NextPosition**;*

*El número de puntos calculados por el algoritmo de calculo_camino se almacena en el puntero *k;*

*La distancia de seguridad, por debajo de la que se considera que los sensores han medido un obstáculo cercano se almacena en **security**;*

printf(Angulo: %lf, Distancia %distancia.);

Se calcula el ángulo de giro entre el ángulo que tiene el robot y el ángulo que forma con el eje vertical el segmento que une las coordenadas actuales del robot con las coordenadas objetivo.

motorRotateAngle(angulo calculado, actualPosition);

La propia función modifica las coordenadas del robot a partir de las cuentas que ha recorrido el robot durante el giro.

Se calcula el módulo de la distancia del segmento que une las coordenadas actuales del robot con las coordenadas objetivo.

motorMoveRelative(distancia entre puntos, actualPosition);

La propia función modifica las coordenadas del robot a partir de las cuentas que ha recorrido el robot durante el movimiento.

La función motorMoveRelative() termina si se recorre la distancia entre puntos o si se detecta un obstáculo.

dist -> distancia medida por los sensores de proximidad.

if distancia medida < security **then**

printf(Obstáculo detectado);

k->0; break;

Si k=0, calculo_camino() no tendrá más puntos que recorrer dentro de camino.

Si esta variable se anula el algoritmo recalculará una nueva trayectoria teniendo en cuenta el obstáculo detectado.

end

else if |NextPosition - ActualPosition| > 100 mm **then**

 genrateInstuctions(ActualPosition, NextPosition, security, k);

end

Fin del algoritmo;

Mediante estas dos funciones auxiliares, calculo_camino() es capaz de recalcular una nueva trayectoria cada vez que se detecta un nuevo obstáculo. Cabe destacar que este algoritmo es reaplicable al experimento anterior simplemente si la variable de distancia de seguridad se anula. De esta forma en ningún caso se elimina la trayectoria inicialmente calculada ni se modifica el mapa con un nuevo obstáculo.

Consideraciones adicionales.

Existen funciones, desarrolladas en la práctica anterior, para implementar el seguimiento de trayectorias, que no se han comentado en este documento pero merecen unas consideraciones adicionales. En el pasado la función motorMoveRelative() implementaba un algoritmo de esquivas de obstáculos. Esta función, cuando el móvil ya había esquivado el obstáculo, recalculaba las instrucciones para alcanzar una posición objetivo teniendo en cuenta que la posición del robot había cambiado. Actualmente ya no es así y toda esta lógica de recálculo de trayectoria se implementa dentro de generateInstructions(). Tras esta actualización, si se cumple la condición de detección de obstáculo la función simplemente termina y actualiza las coordenadas a las coordenadas que tuviera en el momento de la detección el móvil.

1. Anexo

En esta sección se incluye el código en crudo utilizado para esta sesión:

```
esquiva_vertical(Punto* punto_medio,Punto* final,Mapper* mapa,int delta_y){ //caso en el cua

//int origen_y=punto_medio->y;

if(delta_y<0){
    //Escaneo para obtener el punto de contacto
    for(int i=punto_medio->y; mapa->mapa[(i+1)*mapa->columnas+punto_medio->x]==0;i--){
        punto_medio->y=i; //Vamos actualizando verticalmente hasta encontrar el borde del ob

    }
    for(int j=punto_medio->x; j<mapa->columnas;j++){
        if(mapa->mapa[punto_medio->y*mapa->columnas+j]==2){
            //Se encontro un vertice

            punto_medio->x=j;
            break;}
    }
}

else if (delta_y>0){
    //Escaneo para obtener el punto de contacto
    for(int i=punto_medio->y; mapa->mapa[(i-1)*mapa->columnas+punto_medio->x]==0;i++){
        punto_medio->y=i; //Vamos actualizando verticalmente hasta encontrar el borde del ob
    }
    for(int j=punto_medio->x; j<mapa->columnas;j++){
        if(mapa->mapa[punto_medio->y*mapa->columnas+j]==2){
            //Se encontro un vertice
            punto_medio->x=j;
            break;}
    }
}

}

}

void esquiva_horizontal(Punto* punto_medio,Punto* final,Mapper* mapa,int delta_x){ //caso en el

//int origen_y=punto_medio->y;

if(delta_x<0){
    //Escaneo para obtener el punto de contacto
    for(int i=punto_medio->x; mapa->mapa[punto_medio->y*mapa->columnas+i+1]==0;i--){
        punto_medio->x=i; //Vamos actualizando verticalmente hasta encontrar el borde del ob
    }
    for(int j=punto_medio->y; j<mapa->filas;j++){
        if(mapa->mapa[j*mapa->columnas+punto_medio->x]==2){
            //Se encontro un vertice
            punto_medio->y=j;
            break;}
    }
}

}
```

```

    else if (delta_x>0){
        //Escaneo para obtener el punto de contaco
        for(int i=punto_medio->x; mapa->mapa[punto_medio->y*mapa->columnas+i-1]==0;i++){
            punto_medio->x=i; //Vamos actualizando verticalmente hasta encontrar el borde del ob
        }
        for(int j=punto_medio->y; j<mapa->filas;j++){
            if(mapa->mapa[j*mapa->columnas+punto_medio->x]==2){
                //Se encontro un vertice
                punto_medio->y=j;
                break;}
        }
    }
}

/*void motorDodgeRoutine(Position *actualPosition, void *args_ptr, Mapper *mapa){
motorRotateAngle(-90, args_ptr, actualPosition);
motorMoveRelative(0, 400, args_ptr, actualPosition);
motorRotateAngle(90, args_ptr, actualPosition);
motorMoveRelative(0, 500, args_ptr, actualPosition);
Punto dim = {4, 4}
modifyMapper(mapa,actualPosition,&dim);
}*/

void generateInstructions(Position *actualPosition, Position *nextPosition, void *args_ptr, int
printf("Angle1: %f\nDist: %f\n",atan2(nextPosition->x-actualPosition->x, nextPosition->y-actualP
motorRotateAngle(atan2(nextPosition->x-actualPosition->x, nextPosition->y-actualPosition->y)*180
delay(1000);
motorMoveRelative(security, sqrt((nextPosition->x-actualPosition->x)*(nextPosition->x-actualPosi

ThreadArgs *data = (ThreadArgs*) args_ptr;
float distLeft, distRight;

pthread_mutex_lock(&data->mutexLeft);
distLeft = data->distLeft;
pthread_mutex_unlock(&data->mutexLeft);

pthread_mutex_lock(&data->mutexRight);
distRight = data->distRight;
pthread_mutex_unlock(&data->mutexRight);

if(distLeft <= security || distRight <= security){
printf("Obstaculo cercano detectado(Rht: %f, Lft: %f)\n",distRight,distLeft);
*k=0;
pwmWrite(Pin_I,300);
pwmWrite(Pin_D,300);
delay(1000);

}

else if(sqrt((actualPosition->x-nextPosition->x)*(actualPosition->x-nextPosition->x) + (actualPo
printf("Generando de nuevo instrucciones\n");
generateInstructions(actualPosition, nextPosition, args_ptr, security, k);}

```

```
}
```

```
int funcion_aux_tipo_movimiento(Punto *origen,Punto* final,Punto* punto_medio,Mapper* mapa){
//Esta funcion determinara si hay un objeto en el caso de
    int delta_x0=final->x-origen->x;
    int delta_y0=final->y-origen->y;
    int deteccion_obstaculo=0;

    if(delta_x0==0 && delta_y0<0){

        for(int i=origen->y;i>final->y;i--) //barrido en filas
        {

            if ((mapa->mapa[i*mapa->columnas+origen->x]==0)&&(mapa->mapa[(i-1)*mapa->columnas+origen->x]==0))
            {
                deteccion_obstaculo=1;//vale 1 cuando se detecta un objeto atravesado verticalmente
            }

        }

    }

    else if(delta_x0==0 && delta_y0>0){
for(int i=origen->y;i<final->y;i++){ //barrido en filas

        if ((mapa->mapa[i*mapa->columnas+origen->x]==0)&&(mapa->mapa[(i+1)*mapa->columnas+origen->x]==0))
        {
            deteccion_obstaculo=1;//vale 1 cuando se detecta un objeto atravesado verticalmente
        }

    }

}

else if(delta_y0==0 && delta_x0>0){
for(int i=origen->x;i<final->x;i++){ //barrido en filas

        if ((mapa->mapa[origen->y*mapa->columnas+i]==0)&&(mapa->mapa[origen->y*mapa->columnas+i+1]==0))
        {
            deteccion_obstaculo=2;//vale 2 cuando se detecta un objeto atravesado horizontalmente
        }

    }

}

else if(delta_y0==0 && delta_x0<0){
for(int i=origen->x;i>final->x;i--){ //barrido en filas

        if ((mapa->mapa[origen->y*mapa->columnas+i]==0)&&(mapa->mapa[origen->y*mapa->columnas+i-1]==0))
        {
            deteccion_obstaculo=2;//vale 2 cuando se detecta un objeto atravesado horizontalmente
        }

    }

}
```

```

    }

}

return deteccion_obstaculo;
}

float* calculo_camino(Punto *origen,Punto* final,Punto* punto_medio,Mapper* mapa,int* N, void *
    printf("Cálculo de camino\n");
    int contador_vertices;
    int delta_x, delta_y;
    int k=0;
    float* camino=NULL;//inicializamos el puntero NULL
    //Hacemos una primera reserva de memoria
    camino=(float*)(malloc(2*sizeof(float)));
    //Se buscan los dos proximos vertices y se evalua cual es mas optimo
    int move_type=funcion_aux_tipo_movimiento(origen, final, punto_medio,mapa);

    //Hay que implementar algoritmo que calcule el proximo punto y lo actualice
    while (punto_medio->x != final->x || punto_medio->y != final->y) { //Mientras no estemos
    //Volvemos a
    delta_x=final->x-punto_medio->x;
    delta_y=final->y-punto_medio->y;
    contador_vertices=0;

    if((delta_x<=0)&&(delta_y<=0)) {
    //En este caso estamos por encima y a la derecha del punto que se desea alcanzar
    if(move_type==1){ //Si
        //Rutina de esquiiva vertical
        esquiiva_vertical(punto_medio,final,mapa,delta_y);
        contador_vertices++;
        k+=2;
        camino=save_point(camino,k,punto_medio);
        printf("Se ha actualizado el punto medio (%d, %d).\n",punto_medio->x,12-punto_medio->y);
        move_type=0;

        //delay(2000);
    }

    else if(move_type==2){//Rutina esquiiva horizontal
        esquiiva_horizontal(punto_medio,final,mapa,delta_x);
        contador_vertices++;
        k+=2;
        camino=save_point(camino,k,punto_medio);
        printf("Se ha actualizado el punto medio (%d, %d).\n",punto_medio->x,12-punto_medio->y);
        move_type=0;
        //delay(2000);
    }

    }

    else{

```

```

for(int i=punto_medio->y;i>=0;i--) //barrido en filas
{
    for(int j=punto_medio->x;j>=final->x-5;j--){//barrido en columnas

        if ((mapa->mapa[i*mapa->columnas+j]==0)&&(mapa->mapa[i*mapa->columnas+j-1]==1))
        {//salida del bucle si se detecta un objeto siendo atravesado horizontalmente
            break;
        }
        else if((mapa->mapa[i*mapa->columnas+j]==0)&&(mapa->mapa[(i-1)*mapa->columnas+j]==1))
        {
            break;
        }
        else if((mapa->mapa[i*mapa->columnas+j]==2)&&(j!=punto_medio->x || i!=punto_medio->y))
        {
            k=optimizacion_vertices(j,i,final,punto_medio,k,&contador_vertices);
            camino=save_point(camino,k,punto_medio);
        }
    }
}

}

else if((delta_x<=0)&&(delta_y>=0)) {

if(move_type==1){
    //Rutina de esquiva vertical
    esquiva_vertical(punto_medio,final,mapa,delta_y);
    contador_vertices++;
    k+=2;
    camino=save_point(camino,k,punto_medio);
    printf("Se ha actualizado el punto medio (%d, %d).\n",punto_medio->x,12-punto_medio->y);
    move_type=0;

    //delay(2000);
}

else if(move_type==2){//Rutina esquiva horizontal
    esquiva_horizontal(punto_medio,final,mapa,delta_x);
    contador_vertices++;
    k+=2;
    camino=save_point(camino,k,punto_medio);
    printf("Se ha actualizado el punto medio (%d, %d).\n",punto_medio->x,12-punto_medio->y);
    move_type=0;

    //delay(2000);
}
//En este caso estamos por debajo y a la derecha del punto que se desea alcanzar
else
{for(int i=punto_medio->y;i<=mapa->filas-1;i++) //barrido en filas
{ // printf("Iteraacion %d\n", i);
    for(int j=punto_medio->x;j>=final->x-5;j--){//barrido en columnas

        if ((mapa->mapa[i*mapa->columnas+j]==0)&&(mapa->mapa[i*mapa->columnas+j-1]==1))
        {//salida del bucle si se detecta un objeto siendo atravesado horizontalmente

```

```

        //printf("Bloqueo atravesar horizontalmente\n");
        break;
    }
    else if((mapa->mapa[i*mapa->columnas+j]==0)&&(mapa->mapa[(i+1)*mapa->columnas+j]
        //printf("Bloqueo atravesar verticalmente\n");
        break;
    }
    else if((mapa->mapa[i*mapa->columnas+j]==2)&&(j!=punto_medio->x || i!=punto_medio->y))
    {
        k=optimizacion_vertices(j,i,final,punto_medio,k,&contador_vertices);
        //printf("Detectamos el vertice (%d,%d)\n",j,12-i);
        camino=save_point(camino,k,punto_medio);
    }
}
}
}

else if ((delta_x>=0)&&(delta_y>=0)) //Estamos aqui
{
    if(move_type==1){
        //Rutina de esquiua vertical
        esquiua_vertical(punto_medio,final,mapa,delta_y);
        contador_vertices++;
        k+=2;
        camino=save_point(camino,k,punto_medio);
        printf("Se ha actualizado el punto medio (%d, %d).\n",punto_medio->x,12-punto_medio->y);
        move_type=0;

        //delay(2000);
    }

    else if(move_type==2){//Rutina esquiua horizontal
        esquiua_horizontal(punto_medio,final,mapa,delta_x);
        contador_vertices++;
        k+=2;
        camino=save_point(camino,k,punto_medio);
        printf("Se ha actualizado el punto medio (%d, %d).\n",punto_medio->x,12-punto_medio->y);
        move_type=0;

        //delay(2000);
    }
}
//En este caso estamos por debajo y a la izquierda del punto que se desea alcanzar
else {

    for(int i=punto_medio->y;i<=mapa->filas-1;i++) //barrido en filas
    {
        for(int j=punto_medio->x;j<=final->x+5;j++){//barrido en columnas

            if ((mapa->mapa[i*mapa->columnas+j]==0)&&(mapa->mapa[i*mapa->columnas+j+1]==1))
            {
                //salida del bucle si se detecta un objeto siendo atravesado horizontalmente

                break;
            }
        }
    }
}

```

```

    }
    else if((mapa->mapa[i*mapa->columnas+j]==0)&&(mapa->mapa[(i+1)*mapa->columnas+j]
        break;
    }
    else if((mapa->mapa[i*mapa->columnas+j]==2)&&(j!=punto_medio->x || i!=punto_medio->y))
        //printf("Hemos entrado\n");
    k=optimizacion_vertices(j,i,final,punto_medio,k,&contador_vertices);
    camino=save_point(camino,k,punto_medio);
}
}
}
}

else if ((delta_x>=0)&&(delta_y<=0))
{
if(move_type==1){
    //Rutina de esquiava vertical
    esquiava_vertical(punto_medio,final,mapa,delta_y);
    contador_vertices++;
    k+=2;
    camino=save_point(camino,k,punto_medio);
    printf("Se ha actualizado el punto medio (%d, %d).\n",punto_medio->x,12-punto_medio->y);
    move_type=0;

    //delay(2000);
}

else if(move_type==2){//Rutina esquiava horizontal
    esquiava_horizontal(punto_medio,final,mapa,delta_x);
    contador_vertices++;
    k+=2;
    camino=save_point(camino,k,punto_medio);
    printf("Se ha actualizado el punto medio (%d, %d).\n",punto_medio->x,12-punto_medio->y);
    move_type=0;

    //delay(2000);
}
//En este caso estamos por arriba y a la izquierda del punto que se desea alcanzar
else {
for(int i=punto_medio->y;i>=0;i--) //barrido en filas
{
    for(int j=punto_medio->x;j<=final->x+5;j++){//barrido en columnas

        if ((mapa->mapa[i*mapa->columnas+j]==0)&&(mapa->mapa[i*mapa->columnas+j+1]==1))
            //salida del bucle si se detecta un objeto siendo atravesado horizontalmente
            break;
        }
        else if((mapa->mapa[i*mapa->columnas+j]==0)&&(mapa->mapa[(i-1)*mapa->columnas+j]
            break;
        }
        else if((mapa->mapa[i*mapa->columnas+j]==2)&&(j!=punto_medio->x || i!=punto_medio->y))
            //printf("Hemos entrado\n");
            k=optimizacion_vertices(j,i,final,punto_medio,k,&contador_vertices);
            camino=save_point(camino,k,punto_medio);
    }
}
}
}
}

```

```

        k=optimizacion_vertices(j,i,final,punto_medio,k,&contador_vertices);
        camino=save_point(camino,k,punto_medio);
    }
}
}
}

if (contador_vertices==0)
{
    k+=2;
    printf("No se han detectado vertices.\n");
    printf("Se ira directamente al punto final: (%d,%d), partiendo de (%d,%d).\n",final-
    camino=save_point(camino,k,final);
    punto_medio->x=final->x;//
    punto_medio->y=final->y;//Se actualiza condicion salida bucle
    //delay(3000);

}

}

int i=0;

Position *nextPosition = definePosition();
//modifyCoordinates(origen->x*25,(12-origen->y)*25,0, actualPosition);
do{
modifyCoordinates(camino[i],camino[i+1],0, nextPosition);
i+=2;
generateInstructions(actualPosition, nextPosition, args_ptr, safety, &k);
} while(i<k && k!=0);

if(k==0){
origen->x = actualPosition->x/100;
origen->y = actualPosition->y/100;
Punto *dim = create_point(4,4);
modifyMapper(mapa, origen, dim);
free(dim);
//motorRotateAngle(-90, args_ptr, actualPosition);
origen->x = actualPosition->x/100;
origen->y = 12 - actualPosition->y/100;
printf("Se encontro un objeto el mapa modificado es el siguiente:\n");
delay(2000);
printf("Datos almacenados en el mapa cargado:\n");
for (int i = 0; i < mapa->filas*mapa->columnas; i++) {
    if(i%mapa->columnas==0 && i>0) printf("\n");
    printf("%d ", mapa->mapa[i]);
}
printf("\n");
//free(actualPosition);

```

```

free(nextPosition);
*N=0;
calculo_camino(origen, final, origen, mapa, N, args_ptr, actualPosition,0);}
else{
    printf("En principio ya se ha alcanzado el punto objetivo.\n");
    *N=k;
    //free(actualPosition);
    free(nextPosition);}
    return camino;

}

void chargeMap(Mapper *mapa){
    FILE *archivo;
    char buffer[100];
    mapa->filas = 0;
    mapa->columnas = 0;

    archivo = fopen("map.txt", "r");

    if (archivo == NULL) {
        printf("No se pudo abrir el archivo.\n");
        exit(EXIT_FAILURE);
    }

    while (fgets(buffer, sizeof(buffer), archivo)) {
        char *ptr = buffer;
        while (*ptr != '\0') {
            while (isspace(*ptr)) {
                ptr++;
            }
            if (isdigit(*ptr)) {
                mapa->columnas++;
                while (*ptr != '\0' && !isspace(*ptr)){ptr++;}
            } else {
                ptr++;
            }
        }
        mapa->filas++;
    }
    mapa->columnas=mapa->columnas/mapa->filas;
    printf("columnas: %d, filas: %d\n",mapa->columnas, mapa->filas);
    fseek(archivo, 0, SEEK_SET);

    mapa->mapa = (int *)malloc(mapa->filas*mapa->columnas * sizeof(int));

    for (int i = 0; i < mapa->filas*mapa->columnas; i++) {
        if (fscanf(archivo, "%d", &mapa->mapa[i]) != 1) {
            printf("Error al leer los datos del archivo.\n");
            fclose(archivo);
            free(mapa->mapa);
            exit(EXIT_FAILURE);
        }
    }
}

```

```

    }

    fclose(archivo);

    printf("Datos almacenados en el mapa cargado:\n");
    for (int i = 0; i < mapa->filas*mapa->columnas; i++) {
        if(i%mapa->columnas==0 && i>0) printf("\n");
        printf("%d ", mapa->mapa[i]);
    }
    printf("\n");
}

Mapper *defineMapper(){
    Mapper *r = malloc(sizeof(Mapper));
    if (r == NULL) {
        printf("Error: No se pudo asignar memoria para Point3D.\n");
        exit(EXIT_FAILURE);
    }
    chargeMap(r);
    return r;
}

void *movement(void *args_ptr){
    calDistance(args_ptr);

    pwmWrite(Pin_I,300);
    pwmWrite(Pin_D,300);

    ThreadArgs *data = (ThreadArgs*)args_ptr;
    pthread_barrier_wait(&data->barrera);

    /**Experimento 3 **/
    Mapper* mapa = defineMapper();
    int N=0;
    int startX, startY, goalX, goalY;
    printf("Ingrese las coordenadas de inicio (x y): ");
    scanf("%d %d", &startX, &startY);
    startY=mapa->filas-1-startY;
    if((mapa->mapa[startY*mapa->columnas+startX])!=0){
        printf("Ese punto no es válido pues se encuentra en un objeto.\n");
        printf("Ingrese de nuevo las coordenadas de inicio (x y): ");
        scanf("%d %d", &startX, &startY);
        startY=mapa->filas-1-startY;}
    printf("Ingrese las coordenadas del objetivo (x y): ");
    scanf("%d %d", &goalX, &goalY);
    goalY=12-goalY;
    if((mapa->mapa[goalY*mapa->columnas+goalX])!=0){
        printf("Ese punto no es válido pues se encuentra en un objeto.\n");
        printf("Ingrese de nuevo las coordenadas del objetivo (x y): ");
        scanf("%d %d", &goalX, &goalY);
        goalY=mapa->filas-1-goalY;}

```

```

Punto* origen=create_point(startX,startY);
Punto* punto_medio=create_point(startX,startY);
Punto* final=create_point(goalX,goalY);
Position* actualPosition = definePosition();
modifyCoordinates(origen->x*100,(12-origen->y)*100,90, actualPosition);

float* camino=calculo_camino(origen,final,punto_medio,mapa,&N, args_ptr,actualPosition,10);
printf("Puntos del camino: {");
for (int i = 0; i <N; i+=2)
{
    printf(" (%f, %f)",camino[i],camino[i+1]);
}
printf("}\n");

free(actualPosition);
free(mapa->mapa);
free(mapa);
free(origen);
free(punto_medio);
free(final);

motorStop();
data->salir = 1;
pthread_exit(NULL);
return (void *) NULL;
}

int main(int argc, char *argv[]) {

ThreadArgs *sharedData = initSharedData();
//SharedData *sharedData = initSharedData();
int rc;

    wiringPiSetup();

    pinMode(Pin_D,PWM_OUTPUT);
pinMode(Pin_I,PWM_OUTPUT);
pwmSetMode(PWM_MODE_MS);
pwmSetRange(4000);
pwmSetClock(96);

    wiringPiSPISetup(spi_channel,velocidad); //Configuramos canal de comunicación y velocidad
mcp3004Setup (100, spi_channel);
//mcp3004Setup (pbDstRth, spi_channel);

pthread_t thread1,thread2,thread3,thread4,thread5;

    if ((rc=pthread_create(&thread1,NULL,&distanceMeasureRigth,sharedData))) {
printf("Fallo de pthread:\n");
    }
    if ((rc=pthread_create(&thread2,NULL,&distanceMeasureLeft,sharedData))) {

```

```
printf("Fallo de pthread:\n");
    }
    if ((rc=pthread_create(&thread3,NULL,&encoderLeft,sharedData))){
printf("Fallo de pthread:\n");
    }
    if ((rc=pthread_create(&thread4,NULL,&encoderRigth,sharedData))){
printf("Fallo de pthread:\n");
    }
    if ((rc=pthread_create(&thread5,NULL,&movement,sharedData))){
printf("Fallo de pthread:\n");
    }

    pthread_join(thread1,NULL);
    pthread_join(thread2,NULL);
    pthread_join(thread3,NULL);
    pthread_join(thread4,NULL);
    pthread_join(thread5,NULL);
    printf("From main proces IC: %d\n",(int)getpid());

destroySharedData(sharedData);

    return 0;
}
```